

Key Objectives

- Learn the complete **Machine Learning product lifecycle**.
- Focus on practical implementation rather than only algorithms.
- Build industry-ready Machine Learning skills.

Topics Covered

- Data pre-processing
- Model selection
- Model training
- Model deployment
- End-to-end Machine Learning workflow

Machine Learning Explained

Unlike traditional programming, Machine Learning enables systems to **learn patterns from data automatically** instead of following explicitly programmed rules. This makes it effective for tasks such as image classification and spam detection.

Importance

- Growth in data availability and computing power has accelerated Machine Learning adoption.
- Machine Learning professionals are in high demand across industries.
- The course is designed to bridge the gap between theoretical concepts and real-world applications.

Conclusion

The series provides a **structured, day-by-day roadmap** that equips learners with practical Machine Learning knowledge and prepares them for industry roles.

The series provides a structured learning path that helps learners develop practical Machine Learning skills and become industry-ready.

AI, ML & DL – Short Notes

Artificial Intelligence (AI) is the broadest field that aims to build machines capable of performing tasks that normally require human intelligence. Early AI systems, called **Symbolic AI** or **Expert Systems**, worked using manually written rules created by programmers.

Machine Learning (ML) is a subset of AI where computers learn patterns directly from data instead of following explicitly programmed rules. The model is trained on data and automatically learns the relationships needed to make predictions or decisions.

Deep Learning (DL) is a specialized subset of Machine Learning that uses **Artificial Neural Networks (ANNs)** inspired by the human brain. Unlike traditional ML, Deep Learning automatically extracts

important features from raw data, reducing the need for manual feature engineering. It also improves significantly as the amount of training data increases.

Key Difference

- **AI:** Builds intelligent systems.
- **ML:** Learns patterns from data.
- **DL:** Uses neural networks to automatically learn complex patterns from large datasets.

Advantages of Deep Learning

- Automatically extracts features.
- Performs well on image, speech, and text data.
- Accuracy improves with more data.

When to Use Traditional ML

Traditional Machine Learning is preferred when the dataset is small, the problem is simple, or faster training and lower computational resources are required.

AI, ML & DL – Short Notes

Artificial Intelligence (AI) is the broadest field that aims to build machines capable of performing tasks that normally require human intelligence. Early AI systems, called **Symbolic AI** or **Expert Systems**, worked using manually written rules created by programmers.

Machine Learning (ML) is a subset of AI where computers learn patterns directly from data instead of following explicitly programmed rules. The model is trained on data and automatically learns the relationships needed to make predictions or decisions.

Deep Learning (DL) is a specialized subset of Machine Learning that uses **Artificial Neural Networks (ANNs)** inspired by the human brain. Unlike traditional ML, Deep Learning automatically extracts important features from raw data, reducing the need for manual feature engineering. It also improves significantly as the amount of training data increases.

Key Difference

- **AI:** Builds intelligent systems.
- **ML:** Learns patterns from data.
- **DL:** Uses neural networks to automatically learn complex patterns from large datasets.

Advantages of Deep Learning

- Automatically extracts features.
- Performs well on image, speech, and text data.
- Accuracy improves with more data.

When to Use Traditional ML

Traditional Machine Learning is preferred when the dataset is small, the problem is simple, or faster training and lower computational resources are required.

This video by *CampusX* provides a comprehensive introduction to the **four main types of Machine Learning (ML)** based on the level of external supervision required during the training process (0:45).

1. Supervised Machine Learning (1:50): This approach uses a dataset that includes both inputs and outputs (labeled data) to teach the algorithm to predict results for new, unseen input. It is categorized into two main types:

- **Regression (4:40):** Used when the output is a numerical value (e.g., predicting salary or house prices).
- **Classification (4:47):** Used when the output is categorical (e.g., predicting 'yes' or 'no' for student placements or spam detection).

2. Unsupervised Learning (8:10): Here, the algorithm is provided only with input data and no specific labels. It focuses on finding hidden patterns or structures in the data through:

- **Clustering (10:05):** Grouping data points based on similarities (e.g., customer segmentation).
- **Dimensionality Reduction (12:32):** Simplifying complex datasets with many variables into a more manageable format while keeping essential information (e.g., *PCA*).
- **Anomaly Detection (17:07):** Identifying unusual data points that deviate from the normal pattern (e.g., credit card fraud detection).
- **Association Rule Learning (18:11):** Discovering rules that describe large portions of data, such as the famous *Beer and Diaper* correlation, to optimize product placement in stores.

3. Semi-Supervised Learning (20:55): This technique uses a small amount of labeled data combined with a large amount of unlabeled data. It is highly useful because labeling data can be expensive and time-consuming (e.g., *Google Photos* identifying people in pictures after you label just one photo).

4. Reinforcement Learning (23:17): In this model, an *agent* learns to make decisions through trial and error within an environment to maximize rewards and minimize punishment (like training a model to play complex games such as *Go* via *AlphaGo*).

Batch Machine Learning – Short Notes

Batch Machine Learning (Offline Learning) is a training approach in which the model is trained **once on the entire dataset** before deployment. After training, the model is tested and deployed to the **production environment**, where it serves real users. Since the deployed model is **static**, it cannot learn from new incoming data automatically. To keep the model updated, it must be **retrained periodically** (daily, weekly, or monthly) using the latest data and then redeployed.

A **production environment** is the real-world server or system where the trained machine learning model is used by customers to make predictions.

Advantages of Batch Learning

- Easy to implement.
- Suitable for stable datasets.
- Produces high accuracy when trained on complete data.
- Widely used in many machine learning applications.

Disadvantages of Batch Learning

- Cannot learn from new data in real time.
- Requires periodic retraining and redeployment.
- Difficult to handle very large datasets.
- Not suitable for devices with limited internet connectivity.
- Cannot quickly adapt to changing trends or business conditions.

Applications

- House price prediction
- Salary prediction
- Medical diagnosis
- Customer churn prediction
- Credit risk analysis

Quick Revision

- **Training:** Entire dataset at once.
- **Learning:** Offline.
- **Updates:** Periodic retraining.
- **Best For:** Static datasets.
- **Not Suitable For:** Real-time or continuously changing data.

Interview Questions & Answers

Q1. What is Batch Machine Learning?

Answer:

Batch Machine Learning, also called **Offline Learning**, is a learning approach where the model is trained using the **entire dataset at once** before deployment. After deployment, the model becomes static and cannot learn from new data until it is retrained.

Q2. What are the advantages and disadvantages of Batch Learning?

Answer:

Advantages: It is simple to implement, works well with stable datasets, and generally provides good accuracy.

Disadvantages: It cannot learn from new data in real time, requires periodic retraining, struggles with very large datasets, and is not suitable for applications where data changes frequently.

Online Machine Learning – Structured Summary

1. Definition

Online Machine Learning is an **incremental learning** approach where the model is trained on **small, sequential batches of data** instead of the entire dataset. The model continues learning while it is deployed in the **production environment**, allowing it to adapt to new data in real time

2. When to Use Online Learning

Online Learning is suitable when:

- Data changes continuously (**Concept Drift**), e.g., stock prices, e-commerce trends.
- Datasets are too large to fit into memory (**Out-of-Core Learning**).
- Applications require **real-time model updates**.

3. Implementation

Common libraries used for Online Learning:

- **Scikit-learn** (partial_fit)
- **River**
- **Vowpal Wabbit**

4. Risks & Challenges

- **Catastrophic Interference:** The model may forget previously learned information while learning new data.
- **Continuous Monitoring:** Required to detect performance degradation.
- **Data Bias:** Malicious or incorrect input data can negatively affect the model.

Offline Learning	Features	Online Learning
Less complex as model is constant	Complexity	Dynamic complexity as the model keeps evolving over time
Fewer computations, single time batch-based training	Computational Power	Continuous data ingestions result in consequent model refinement computations
Easier to implement	Use in Production	Difficult to implement and manage
Image Classification or anything related to Machine Learning - where data patterns remains constant without sudden concept drifts	Applications	Used in finance, economics, health where new data patterns are constantly emerging
Industry proven tools. E.g. Sci-kit, TensorFlow, Pytorch, Keras, Spark Mlib	Tools	Active research/New project tools: E.g. MOA, SAMOA, scikit-multiflow, streamDM

Challenges in Machine Learning – Structured Summary

1. Data Collection

- Collecting high-quality and relevant data is one of the biggest challenges.
- Real-world data collection is often more difficult than building the model.

2. Insufficient Data

- Machine Learning models require large amounts of data.
- Small datasets usually lead to poor accuracy and weak performance.

3. Non-Representative Data

- Training data should represent real-world data.
- Biased or unbalanced data leads to sampling bias and inaccurate predictions.

4. Poor Quality Data

- Missing values, duplicate records, incorrect formats, and noisy data reduce model performance.
- Data cleaning takes a significant amount of a data scientist's time.

5. Irrelevant Features

- Unnecessary features reduce model accuracy.
- Feature engineering helps select the most important features.

6. Overfitting

- The model memorizes the training data instead of learning patterns.
- Performs well on training data but poorly on unseen data.

7. Underfitting

- The model is too simple to learn the underlying patterns.
- Performs poorly on both training and testing data.

8. Software Integration

- Integrating ML models into web, mobile, IoT, or enterprise applications is challenging.

9. Offline Learning & Deployment

- Updating deployed models and maintaining performance requires periodic retraining and monitoring.

10. Cost

- ML projects involve costs for data collection, cloud infrastructure, deployment, monitoring, and maintenance.
 - Large-scale projects often require MLOps for efficient management.
-

Applications of Machine Learning – Structured Summary

1. Retail Sector

- Used for inventory management and stock forecasting.
- Analyzes customer buying patterns.
- Builds customer profiles for personalized marketing and recommendations.

2. Banking and Finance

- Used for loan approval and credit risk analysis.
- Predicts the likelihood of loan default.
- Helps detect fraudulent transactions and reduce financial risks.

3. Transportation

- Used by platforms like Ola and Uber.
- Performs demand forecasting.
- Implements dynamic surge pricing to balance supply and demand.

4. Manufacturing

- Used for predictive maintenance.
- Monitors machine health using sensor data.
- Predicts equipment failures before they occur, reducing downtime.

5. Consumer Internet & Sentiment Analysis

- Analyzes opinions from social media platforms like Twitter.
- Identifies positive, negative, or neutral sentiments.

- Used in politics, marketing, and stock market analysis.
-

Machine Learning Development Life Cycle (MLDLC) – Structured Summary

1. Framing the Problem

- Define the project objective.
- Identify the target audience.
- Decide evaluation metrics and required resources.

2. Gathering the Data

- Collect data from APIs, databases, web scraping, or files.
- Organize and store the collected data.

3. Data Preprocessing

- Remove duplicate records.
- Handle missing values.
- Clean and transform the data.
- Scale or normalize features.

4. Exploratory Data Analysis (EDA)

- Analyze the dataset using statistics and visualizations.
- Find patterns, relationships, and outliers.
- Understand the data before model building.

5. Feature Engineering & Feature Selection

- Create new useful features.
- Select the most important features.
- Improve model accuracy and reduce training time.

6. Model Training, Evaluation & Selection

- Train different machine learning models.
- Evaluate models using performance metrics.
- Tune hyperparameters.
- Select the best-performing model.

7. Model Deployment

- Convert the trained model into an API or deploy it on a server.
- Make the model available for real-world users.

8. Beta Testing

- Release the model to a limited group of users.
- Collect feedback and identify issues.
- Verify model performance in real-world scenarios.

9. Model Optimization

- Continuously monitor the deployed model.
 - Detect data drift or performance degradation.
 - Retrain and update the model when required.
-

Roles in the Data Science & Machine Learning Ecosystem – Structured Summary

1. Data Engineer

- Builds and manages data pipelines.
- Collects, stores, and processes large volumes of data.
- Converts **OLTP (transactional)** data into **OLAP (analytical)** data.
- Ensures high-quality data is available for analysis.
- **Skills:** Python, SQL, Cloud Platforms (AWS/GCP/Azure), Apache Spark, Hadoop, ETL.

2. Data Analyst

- Analyzes historical data to generate business insights.
- Cleans and prepares data.
- Creates dashboards, reports, and visualizations.
- Presents findings to stakeholders for decision-making.
- **Skills:** SQL, Excel, Statistics, Tableau, Power BI.

3. Data Scientist

- Builds predictive and machine learning models.
- Solves complex business problems using data.
- Performs feature engineering, model training, and evaluation.
- Requires strong knowledge of mathematics, statistics, and machine learning.
- **Skills:** Python, Machine Learning, Deep Learning, Statistics, SQL.

4. Machine Learning Engineer

- Deploys machine learning models into production.
- Optimizes and maintains deployed models.
- Bridges the gap between Data Scientists and software applications.
- Focuses on scalability, performance, and system integration.

- **Skills:** Python, MLOps, Docker, Kubernetes, FastAPI, Cloud Platforms, System Design.

Common <code>read_csv()</code> Parameters	
Parameter	Purpose
<code>sep</code>	Specifies the delimiter
<code>header</code>	Defines header row
<code>names</code>	Assigns custom column names
<code>usecols</code>	Selects specific columns
<code>skiprows</code>	Skips rows
<code>nrows</code>	Reads limited rows
<code>dtype</code>	Sets column data types
<code>parse_dates</code>	Converts date columns
<code>converters</code>	Applies custom transformations
<code>na_values</code>	Handles missing values
<code>chunksize</code>	Reads large files in chunks

Working with CSV Files in Pandas – Structured Summary

1. Loading CSV Files

- Used to load CSV files from a local system or a URL.
- **Syntax:**

```
import pandas as pd
```

```
df = pd.read_csv("data.csv")
```

From URL:

```
df = pd.read_csv("https://example.com/data.csv")
```

2. Handling Different Separators

- Used when the file is separated by tabs or other delimiters.

Syntax:

```
df = pd.read_csv("data.tsv", sep="\t")
```

Other separators:

```
sep=","
```

```
sep=";"
```

```
sep="|"
```

3. Custom Column Names

- Assign your own column names.

Syntax:

```
df = pd.read_csv(  
    "data.csv",  
    names=["Name", "Age", "Salary"]  
)
```

4. Reading Files Without Header

- Used when the CSV file has no header row.

Syntax:

```
df = pd.read_csv(  
    "data.csv",  
    header=None  
)
```

5. Selecting Specific Columns

- Import only required columns.

Syntax:

```
df = pd.read_csv(  
    "data.csv",  
    usecols=["Name", "Age"]  
)
```

or

```
df = pd.read_csv(  
    "data.csv",  
    usecols=[0, 2]  
)
```

6. Skipping Rows

- Skip unnecessary rows while reading.

Syntax:

```
df = pd.read_csv(  
    "data.csv",  
    skiprows=2  
)
```

7. Reading Limited Rows

- Load only the first few rows.

Syntax:

```
df = pd.read_csv(  
    "data.csv",  
    nrows=100  
)
```

8. Changing Data Types

- Reduce memory usage by specifying data types.

Syntax:

```
df = pd.read_csv(  
    "data.csv",  
    dtype={"Age": "int32"}  
)
```

9. Parsing Date Columns

- Convert date columns into datetime objects.

Syntax:

```
df = pd.read_csv(  
    "data.csv",  
    parse_dates=["Date"]  
)
```

10. Using Converters

- Apply custom functions while reading data.

Syntax:

```
df = pd.read_csv(  
    "data.csv",  
    converters={  
        "Gender": lambda x: x.upper()  
    }  
)
```

11. Handling Missing Values

- Specify values that should be treated as missing.

Syntax:

```
df = pd.read_csv(  
    "data.csv",  
    na_values=["NA", "?", "Missing"]  
)
```

12. Reading Large Files (Chunks)

- Read very large CSV files in small chunks.

Syntax:

```
chunk = pd.read_csv(  
    "large.csv",  
    chunksize=1000  
)
```

for data in chunk:

```
    print(data.head())
```